

Actividad 2 - Búsqueda y sistemas basados en reglas

Integrantes:

Luz Rodriguez

Juan Sánchez

Nicoll Piza

Yadir Contreras

Facultad de Ingeniería

Universidad Iberoamericana

24082026_C1_202634 Inteligencia artificial

Ing. Sandra Isabel Rodríguez Bautista

20 de septiembre de 2026

Actividad 2 - Búsqueda y sistemas basados en reglas

AUTORES

LUZ CELESTE RODRIGUEZ CABARCAS

JUAN GUILLERMO SÁNCHEZ

NICOLL DAIHANN PIZA

YADIR HERNANDO CONTRERAS

Presentado como actividad 2 de: Inteligencia artificial

DOCENTE

ING. SANDRA ISABEL RODRIGUEZ BAUTISTA

UNIVERSIDAD IBEROAMERICANA

FACULTAD DE INGENIERIA

CARRERA DE INGENIERIA DE SOFTWARE

Bogotá D.C

20 de septiembre de 2026

Tabla de contenido

Introducción	4
Enlaces del Proyecto	5
Repositorio en Git (Código y Pruebas):	5
Video Explicativo (YouTube)	5
Estructura del Repositorio y Código Fuente	5
Descripción de los archivos e historial de versiones:	6
Registro de actividad (Commits):	6
Documentación y Recursos de Apoyo	6
Descripción de los archivos alojados en docs	7
Evidencia de colaboración	7
Pruebas	8
Objetivo de las pruebas	8
Casos de prueba ejecutados	8
Comparación de rutas	10
Verificación contra Dijkstra	10
Conclusiones de las pruebas	11
Conclusiones generales	12
Referencias	14

Introducción

En el contexto de la inteligencia artificial, la representación del conocimiento y las técnicas de búsqueda heurística constituyen pilares fundamentales para la resolución de problemas complejos en la vida real. El presente proyecto aborda el diseño y desarrollo de un sistema inteligente en Python aplicado al ámbito del transporte masivo local. A partir de una base de conocimiento fundamentada en reglas de lógica formal y el uso de algoritmos de búsqueda, el sistema tiene como objetivo principal determinar la ruta más eficiente entre un punto de origen y uno de destino. En este informe (y en el video adjunto), se detallan la estructura de la base de reglas, la lógica del código fuente, las ejecuciones de prueba realizadas y la evidencia del trabajo colaborativo del equipo.

Enlaces del Proyecto

Todos los recursos del proyecto se encuentran alojados en el repositorio oficial. El tutor del curso ha sido agregado como colaborador para la revisión del código fuente y del historial de contribuciones.

Repositorio en Git (Código y Pruebas):

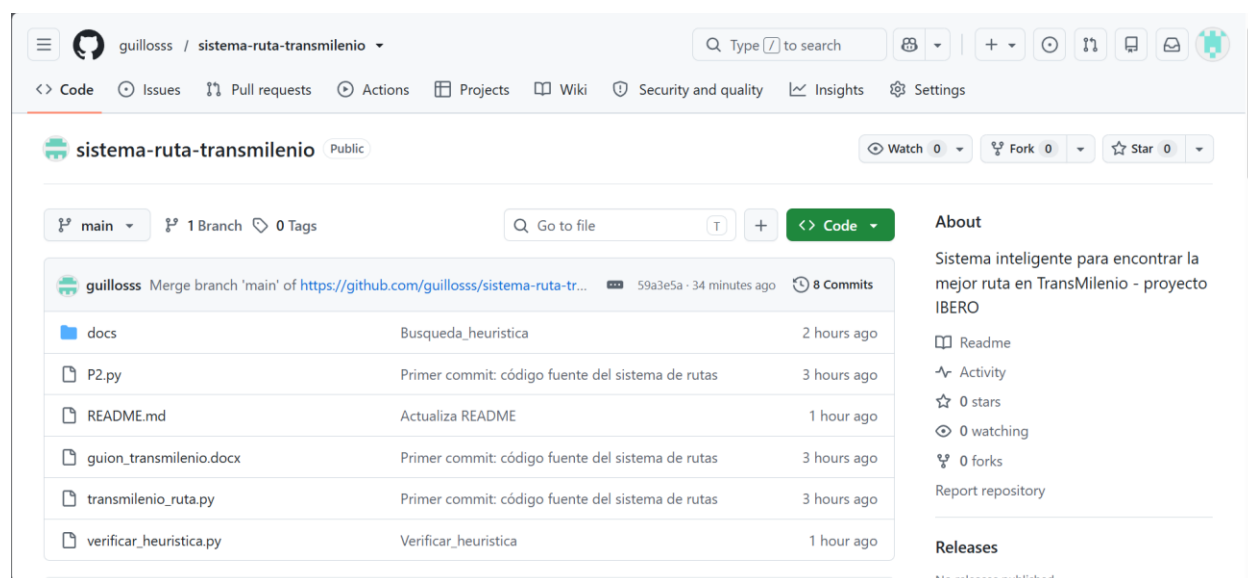
[guilloss/sistema-ruta-transmilenio](https://github.com/guilloss/sistema-ruta-transmilenio): Sistema inteligente para encontrar la mejor ruta en TransMilenio - proyecto IBERO

Video Explicativo (YouTube)

<https://youtu.be/DsewNB06OaE?feature=shared>

Estructura del Repositorio y Código Fuente

A continuación, se presenta la vista principal del repositorio en GitHub (guilloss / sistema-ruta-Transmilenio), donde se aloja el proyecto denominado "Sistema inteligente para encontrar la mejor ruta en Transmilenio



Descripción de los archivos e historial de versiones:

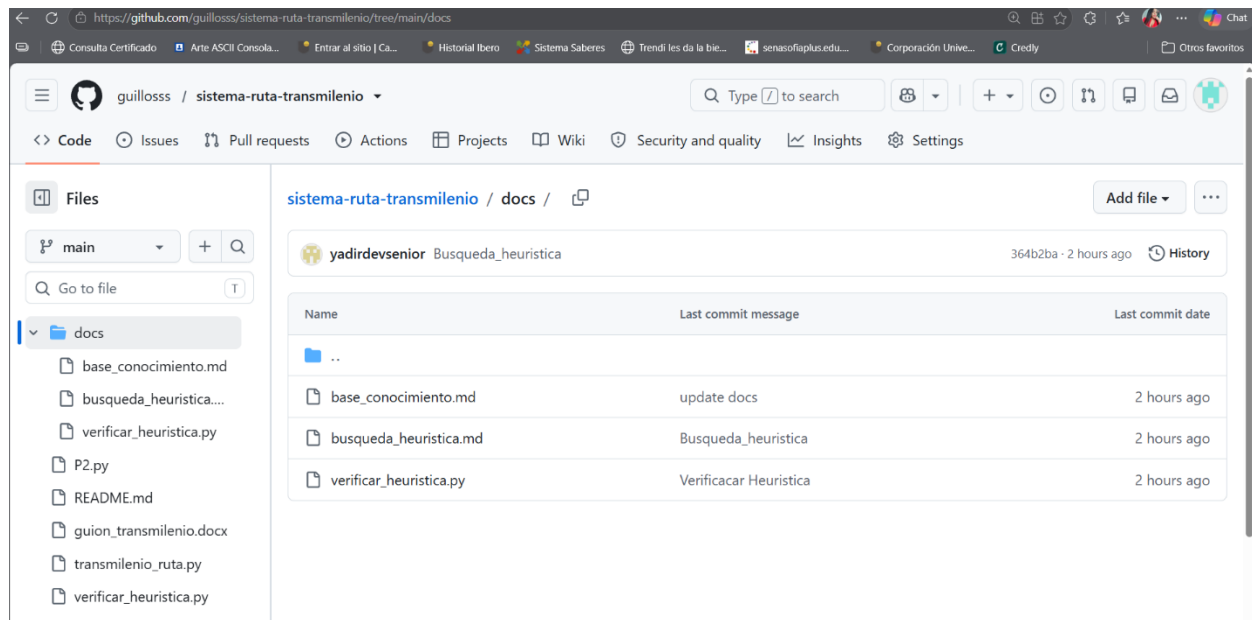
- `transmilenio_ruta.py` & `P2.py`: Archivos que contienen el código fuente en Python para la base de conocimiento y el sistema inteligente de búsqueda de rutas.
- `verificar_heuristica.py`: Script enfocado en la validación de las métricas y técnicas de búsqueda heurística.
- `docs/`: Carpeta que contiene la documentación referente a la búsqueda heurística.
- `guion_transmilenio.docx`: Guion descriptivo utilizado para la grabación del video explicativo.
- `README.md`: Documento explicativo con la descripción general del proyecto e instrucciones de ejecución.

Registro de actividad (Commits):

Se evidencian los commits realizados durante el desarrollo del proyecto (como "Primer commit: código fuente del sistema de rutas", "Busqueda_heuristica", "Verificar_heuristica" y "Actualiza README"), demostrando la subida progresiva y estructurada de los archivos al control de versiones.

Documentación y Recursos de Apoyo

En esta sección del repositorio se evidencia la documentación teórica y técnica del proyecto, así como el trabajo colaborativo:



Descripción de los archivos alojados en docs

- `base_conocimiento.md`: Documento en Markdown que detalla la estructura formal de las reglas lógicas y la base de conocimiento implementada para el sistema de transporte.
- `busqueda_heuristica.md`: Informe técnico que explica las estrategias y métricas de búsqueda heurística empleadas para encontrar la ruta óptima entre dos puntos.
- `verificar_heuristica.py`: Script en Python diseñado para realizar pruebas y validaciones sobre el algoritmo de búsqueda heurística.

Evidencia de colaboración

Se observa los commits realizado por los distintos integrantes, demostrando la división de tareas y el aporte específico en la documentación del modelo teórico y de búsqueda.

Pruebas

Objetivo de las pruebas

El objetivo de esta sección es comprobar que el sistema, implementado con el algoritmo de búsqueda heurística A*, efectivamente encuentra la ruta de menor costo entre dos estaciones de Transmilenio, y que maneja correctamente los casos donde una estación no existe.

Adicionalmente, se comparó el resultado del algoritmo contra el algoritmo de Dijkstra (fuerza bruta) para confirmar que la heurística utilizada es correcta.

Casos de prueba ejecutados

Prueba 1 — Ruta que aprovecha un tramo expreso

Origen: Portal Norte. Destino: Universidad Nacional.

Comando ejecutado:

```
python3 transmilenio_ruta.py "Portal Norte" "Universidad Nacional"
```

Salida obtenida:

SISTEMA INTELIGENTE DE RUTAS - TRANSMILENIO (A*)

Ruta encontrada: Portal Norte -> Toberín -> Calle 100 -> Calle 85 -> Calle 76 -> Marly -> Calle 45 -> Universidad Nacional
Costo total estimado: 21 minutos

Detalle de tramos:

Portal Norte	-> Toberín	(3 min)
Toberín	-> Calle 100	(4 min)
Calle 100	-> Calle 85	(2 min)
Calle 85	-> Calle 76	(2 min)
Calle 76	-> Marly	(5 min)
Marly	-> Calle 45	(2 min)
Calle 45	-> Universidad Nacional	(3 min)

El sistema eligió pasar por el tramo expreso Calle 76 → Marly en lugar de recorrer las estaciones intermedias (Héroes, Calle 72, Calle 63, Calle 57), porque esa alternativa hubiera costado 27 minutos en vez de 21.

Prueba 2 — Descartar un camino más largo

Origen: Héroes. Destino: Universidad Nacional.

```
python3 transmilenio_ruta.py "Héroes" "Universidad Nacional"
```

Salida obtenida:

```
Ruta encontrada: Héroes -> Calle 72 -> Calle 63 -> Calle 57 -> Marly -> Calle 45 ->
Universidad Nacional
Costo total estimado: 13 minutos
```

Aunque existe una conexión por el ramal alternativo de la NQS (Héroes → NQS - Calle 30 → Universidad Nacional), el sistema la descartó porque esa ruta cuesta 17 minutos — 4 minutos más que la opción elegida.

Prueba 3 — Estación inexistente

Origen: Portal Norte. Destino: Estacion Invalida (no existe en la base de conocimiento).

```
python3 transmilenio_ruta.py "Portal Norte" "Estacion Invalida"
```

Salida obtenida:

```
No se encontró una ruta entre 'Portal Norte' y 'Estacion Invalida'.
```

El sistema responde de forma controlada en vez de fallar con un error, lo cual es importante para que sea utilizable por cualquier usuario.

Prueba 4 — Ruta que sí usa el ramal alternativo

Origen: Calle 100. Destino: NQS - Calle 30.

```
python3 transmilenio_ruta.py "Calle 100" "NQS - Calle 30"
```

Salida obtenida:

```
Ruta encontrada: Calle 100 -> Calle 85 -> Calle 76 -> Héroes -> NQS - Calle 30
Costo total estimado: 15 minutos
```

Comparación de rutas

Origen	Destino	Ruta elegida	Costo
Portal Norte	Universidad Nacional	Vía tramo expreso (Calle 76 → Marly)	21 min
Héroes	Universidad Nacional	Vía cadena directa (sin usar la NQS)	13 min
Calle 100	NQS - Calle 30	Vía Héroes (usa el ramal alterno)	15 min
Portal Norte	Estacion Invalida	No existe ruta	—

Verificación contra Dijkstra

Para confirmar que el algoritmo A* no solo encuentra rutas válidas sino las de menor costo, se ejecutó el script verificar_heuristica.py, que compara su resultado contra el algoritmo de Dijkstra (que revisa exhaustivamente todos los caminos posibles) para las 156 combinaciones posibles de origen y destino del grafo.

```
Comparando A* contra Dijkstra para todos los pares de estaciones...
```

```
Comparaciones realizadas: 156
```

```
Diferencias encontradas: 0
```

```
La heurística es admisible: A* siempre encontró el costo óptimo.
```

Los dos algoritmos coincidieron en el 100% de los casos, lo que confirma que el sistema es confiable.

Conclusiones de las pruebas

- Representar el sistema de transporte como una base de conocimiento (estaciones y conexiones con costo) permite automatizar decisiones que a mano tomarían tiempo y serían propensas a error.
- La búsqueda heurística con A* no solo encuentra una ruta válida, sino la de menor costo: en las pruebas realizadas, eligió correctamente el tramo expreso cuando convenía (Prueba 1) y lo descartó cuando no convenía (Prueba 2).
- Diseñar bien la heurística es clave: si sobreestima el costo real, el algoritmo puede fallar. Por eso se verificó matemáticamente contra Dijkstra en las 156 combinaciones posibles del grafo, sin encontrar ninguna diferencia.
- El sistema maneja correctamente los casos inválidos (Prueba 3), lo cual lo hace robusto para uso real.
- Este mismo enfoque podría escalar a todo el mapa real de Transmilenio, agregando más estaciones, líneas y costos reales de tiempo.

Conclusiones generales

El desarrollo e implementación de este sistema inteligente demostró la efectividad de combinar la representación del conocimiento mediante reglas lógicas con algoritmos de búsqueda heurística (específicamente A*) para la resolución de problemas de optimización de rutas en redes de transporte masivo como Transmilenio. A partir de las pruebas realizadas y la validación del sistema, se destacan los siguientes puntos clave:

1. **Eficiencia en la Toma de Decisiones:** Representar el sistema de transporte como una base de conocimiento (grafo de estaciones, tramos y tiempos de desplazamiento) permitió automatizar la búsqueda de trayectos óptimos en tiempo real.
2. **Optimización Heurística Adaptativa:** El algoritmo A* demostró ser capaz de tomar decisiones complejas según la topología de la red:
 - **Aprovechamiento de tramos expresos:** En trayectos largos como Portal Norte →Universidad Nacional, el sistema priorizó el tramo expreso (Calle 76 →Marly) logrando un tiempo total de 21 minutos en lugar de los 27 minutos que tomaría la ruta por estaciones intermedias.
 - **Descarte de rutas más costosas:** En el trayecto Héroes →Universidad Nacional, descartó el ramal alternativo de la NQS por costar 17 minutos, seleccionando la vía directa de 13 minutos.
 - **Uso correcto de ramales secundarios:** En recorridos con destino en líneas de interconexión (Calle 100 →NQS - Calle 30), el sistema utilizó eficientemente el ramal alternativo completando la ruta en 15 minutos.
3. **Validación Formal y Admisibilidad:** Mediante la ejecución del script de verificación (verificar_heuristica.py), se compararon los resultados de A* contra el algoritmo de

Dijkstra en las 156 combinaciones posibles del grafo. Se obtuvo un 0% de discrepancia (100% de coincidencia), lo que demuestra matemáticamente que la función heurística diseñada es admisible y garantiza siempre la ruta de costo mínimo absoluto.

4. Robustez y Escalabilidad: El sistema cuenta con control de errores ante entradas inválidas (por ejemplo, al consultar la ruta hacia una "Estación Invalida"), respondiendo de forma controlada sin interrumpir la ejecución del programa. Esto demuestra que la arquitectura elegida en Python es robusta, mantenible y escalable para integrarle el mapa completo y en tiempo real del sistema de transporte urbano.

Referencias

- Benítez, R. (2014). Inteligencia artificial avanzada. Editorial UOC.
- Nolasco Valenzuela, J. S. (2018). Python: aplicaciones prácticas. RA-MA Editorial.